

Information Retrieval & Retrieval-Augmented Generation

Nhóm 2

Nguyễn Hồng Phúc - 24521390

Vũ Minh Phương - 24521421

Võ Lê Ngọc Thịnh - 24521710

Nguyễn Duy Khang - 24520755

University of Information Technology, VNU-HCM
Faculty of Computer Science
CS221.Q21.KHTN

23/04/2026

- 1 Information Retrieval with Dense Vectors
- 2 Retrieval-Augmented Generation (RAG)
- 3 Datasets
- 4 Evaluating Question Answering

- 1 Information Retrieval with Dense Vectors
- 2 Retrieval-Augmented Generation (RAG)
- 3 Datasets
- 4 Evaluating Question Answering

Hạn chế của các phương pháp dựa trên Sparse Vectors

- Các thuật toán như **tf-idf** và **BM25** dựa trên cơ chế thống kê tần suất từ vựng.
- Chỉ hoạt động dựa trên sự trùng khớp chính xác giữa các từ trong query và document.

Vocabulary mismatch problem

- Người dùng phải dự đoán chính xác thuật ngữ mà tác giả tài liệu đã sử dụng.

Hạn chế của các phương pháp dựa trên Sparse Vectors

- Các thuật toán như **tf-idf** và **BM25** dựa trên cơ chế thống kê tần suất từ vựng.
- Chỉ hoạt động dựa trên sự trùng khớp chính xác giữa các từ trong query và document.

Vocabulary mismatch problem

- Người dùng phải dự đoán chính xác thuật ngữ mà tác giả tài liệu đã sử dụng.

Question

Làm sao để hệ thống hiểu được ý định người dùng khi họ sử dụng các từ đồng nghĩa hoặc cách diễn đạt khác?

The Shift to Dense Retrieval

Giải pháp cho vocabulary mismatch problem

- Chuyển đổi từ các word-count vectors truyền thống sang **dense embeddings**.
- Khả năng xử lý các từ đồng nghĩa (synonymy)
- Lịch sử phát triển:
 - Tiền thân: **Latent Semantic Indexing (LSI)**.
 - Hiện đại: Triển khai thông qua các encoders như BERT để tạo ra các biểu diễn ngữ cảnh.

Cơ chế của kiến trúc:

- Đưa cả query q và document d vào một **single encoder** duy nhất.
- Cơ chế **self-attention** cho phép mọi token của query và document tương tác với nhau.
- Kết quả là một biểu diễn cực kỳ nhạy bén với ngữ nghĩa chung của cặp (q, d) .

Cơ chế của kiến trúc:

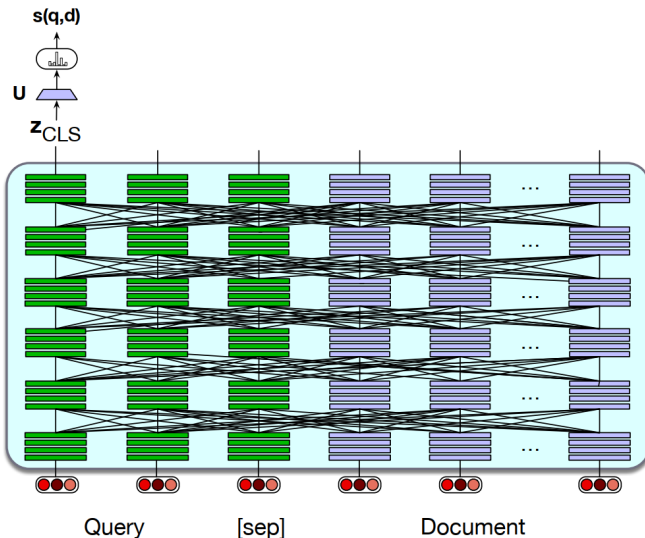
- Đưa cả query q và document d vào một **single encoder** duy nhất.
- Cơ chế **self-attention** cho phép mọi token của query và document tương tác với nhau.
- Kết quả là một biểu diễn cực kỳ nhạy bén với ngữ nghĩa chung của cặp (q, d) .

Tính toán Similarity Score:

- Sử dụng đầu ra của token **[CLS]** làm đại diện cho toàn bộ cặp đầu vào.
- Công thức tính score:

$$\begin{aligned} \mathbf{z} &= \text{BERT}(q; [\text{SEP}]; d)[\text{CLS}] \\ \text{score}(q, d) &= \text{softmax}(U(\mathbf{z})) \end{aligned} \tag{11.17}$$

Cross-Encoder Architecture



Tiền xử lý và Giới hạn kỹ thuật:

- **Passage Chunking:** Chia nhỏ tài liệu thành các đoạn văn ngắn hơn.
- Query và document phải được điều chỉnh để khớp với cửa sổ 512-token của BERT.

Fine-tuning:

- Toàn bộ kiến trúc được tinh chỉnh trên tập dữ liệu chuyên biệt.
- Sử dụng các cặp **relevant** và **non-relevant** để mô hình học cách phân biệt độ liên quan.

Vấn đề của kiến trúc Cross-Encoder:

- Chi phí tính toán và độ trễ lớn.
- Mỗi query mới đòi hỏi phải chạy lại mô hình cho **toàn bộ** tập hợp document.
- Không thể thực hiện tính toán trước biểu diễn của tài liệu vì nó phụ thuộc vào từng query cụ thể.

Kiến trúc mã hóa độc lập:

- Sử dụng hai encoders riêng biệt: $BERT_Q$ cho query và $BERT_D$ cho document.
- **Offline Processing:** Toàn bộ document được mã hóa và lưu trữ dưới dạng **dense vectors** từ trước.

$$\mathbf{z}_d = \text{BERT}_D(d)[CLS]$$

Kiến trúc mã hóa độc lập:

- Sử dụng hai encoders riêng biệt: $BERT_Q$ cho query và $BERT_D$ cho document.
- **Offline Processing:** Toàn bộ document được mã hóa và lưu trữ dưới dạng **dense vectors** từ trước.

$$\mathbf{z}_d = \text{BERT}_D(d)[CLS]$$

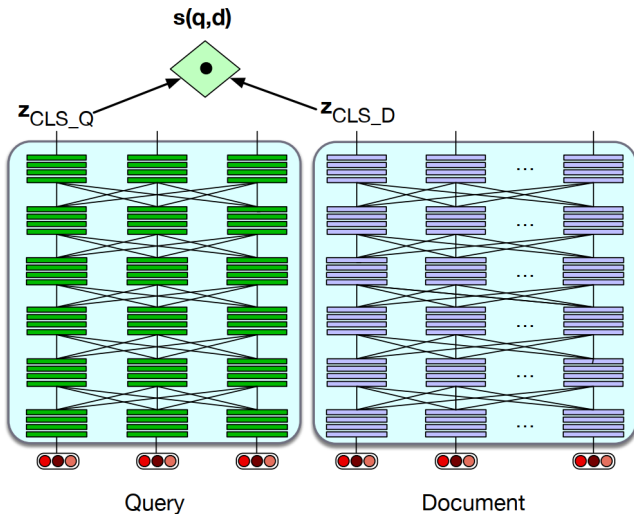
Cơ chế tính toán tại thời điểm truy vấn:

- Khi nhận query, chỉ cần thực hiện **một lần** encode để lấy $\mathbf{z}_q$.
- Score của mỗi document với query được tính toán bằng tích vô hướng:

$$\mathbf{z}_q = \text{BERT}_Q(q)[CLS]$$

$$\text{score}(q, d) = \mathbf{z}_q \cdot \mathbf{z}_d$$

Bi-Encoder Architecture



Trade-off Between Performance and Accuracy:

- Mặc dù rất nhanh, Bi-Encoder ít chính xác hơn so với Full Encoder.
- **Nguyên nhân:** Quyết định về độ liên quan không thể khai thác hết sự tương tác giữa các token trong query và document.
- Mỗi bên (query/document) được biểu diễn độc lập, khiến mô hình khó nhận ra các sắc thái ngữ nghĩa phức tạp phát sinh khi chúng kết hợp với nhau.

Two-Stage Retrieval:

- **Giai đoạn 1 (Retrieval):** Sử dụng các phương pháp nhanh và rẻ (**BM25**) để lọc ra Top N tài liệu tiềm năng nhất từ hàng triệu bản ghi.
- **Giai đoạn 2 (Reranking):** Sử dụng **Full BERT Scoring** để xét kỹ và xếp hạng lại Top N tài liệu đó.

Triết lý thiết kế của ColBERT:

- Thực hiện mã hóa riêng biệt query và document tương tự như Bi-Encoder.
- Giữ lại biểu diễn ngữ cảnh cho từng token sau khi qua BERT.
- Cho phép tính toán trước và lưu trữ biểu diễn của document word để tối ưu hiệu suất.

Triết lý thiết kế của ColBERT:

- Thực hiện mã hóa riêng biệt query và document tương tự như Bi-Encoder.
- Giữ lại biểu diễn ngữ cảnh cho từng token sau khi qua BERT.
- Cho phép tính toán trước và lưu trữ biểu diễn của document word để tối ưu hiệu suất.

Key Concept: Late Interaction

Sự tương tác giữa query và document diễn ra ở bước cuối cùng, giúp cân bằng giữa độ chính xác của Cross-Encoder và tốc độ của Bi-Encoder.

Quy trình xử lý Query q :

- Tokenize thành $[q_1, \dots, q_n]$, chèn thêm $[CLS]$ và token đặc biệt $[Q]$ ở phía trước. Độ dài cố định $N = 32$ tokens (truncate hoặc padding bằng token $[MASK]$).

Quy trình xử lý Query q :

- Tokenize thành $[q_1, \dots, q_n]$, chèn thêm $[CLS]$ và token đặc biệt $[Q]$ ở phía trước. Độ dài cố định $N = 32$ tokens (truncate hoặc padding bằng token $[MASK]$).

Quy trình xử lý Document d :

- Tokenize thành $[d_1, \dots, d_m]$, chèn thêm $[CLS]$ và token đặc biệt $[D]$ ở phía trước. Khác với query, document thường không bị padding cố định để tiết kiệm tài nguyên.

ColBERT: Formal Encoding Process

Quy trình xử lý Query q :

- Tokenize thành $[q_1, \dots, q_n]$, chèn thêm $[CLS]$ và token đặc biệt $[Q]$ ở phía trước. Độ dài cố định $N = 32$ tokens (truncate hoặc padding bằng token $[MASK]$).

Quy trình xử lý Document d :

- Tokenize thành $[d_1, \dots, d_m]$, chèn thêm $[CLS]$ và token đặc biệt $[D]$ ở phía trước. Khác với query, document thường không bị padding cố định để tiết kiệm tài nguyên.

Tối ưu hóa và tạo biểu diễn:

- Áp dụng Linear Layer để (giảm chiều/tối ưu lưu trữ) và Unit Length Rescaling (chuẩn hóa vector), tạo ra hai chuỗi vector cuối cùng E_q và E_d .

ColBERT: Scoring Mechanism (MaxSim)

Late Interaction:

- Điểm số không dựa trên một vector tổng quát mà dựa trên sự tương tác giữa các bộ vector token.
- **Toán tử MaxSim:** Với mỗi token trong query (E_{q_i}), ta tìm token tương đồng nhất trong document (E_{d_j}) thông qua phép tính tích vô hướng.

ColBERT: Scoring Mechanism (MaxSim)

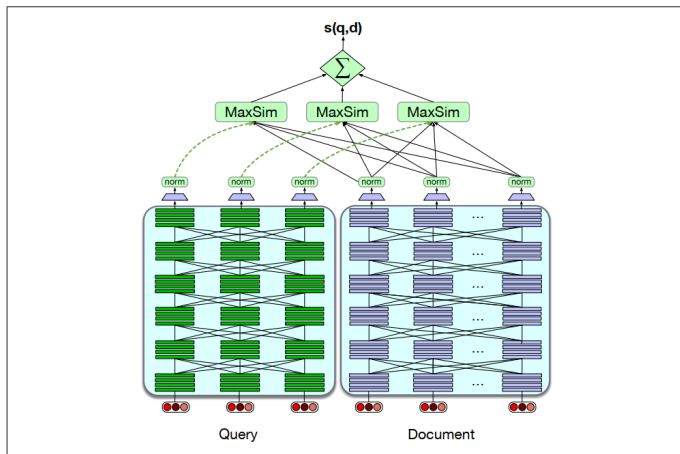
Late Interaction:

- Điểm số không dựa trên một vector tổng quát mà dựa trên sự tương tác giữa các bộ vector token.
- **Toán tử MaxSim:** Với mỗi token trong query (E_{q_i}), ta tìm token tương đồng nhất trong document (E_{d_j}) thông qua phép tính tích vô hướng.

Relevance Score:

$$\text{score}(q, d) = \sum_{i=1}^N \max_{j=1}^m E_{q_i} \cdot E_{d_j} \quad (11.19)$$

ColBERT's Sket



Quy trình huấn luyện End-to-end:

- **Fine-tune BERT:** Cập nhật trọng số của các bộ mã hóa để tạo ra vector đặc trưng tốt hơn.
- **Train from scratch:** Huấn luyện các lớp Linear và các embeddings đặc biệt $[Q]$ và $[D]$.

Quy trình huấn luyện End-to-end:

- **Fine-tune BERT:** Cập nhật trọng số của các bộ mã hóa để tạo ra vector đặc trưng tốt hơn.
- **Train from scratch:** Huấn luyện các lớp Linear và các embeddings đặc biệt $[Q]$ và $[D]$.

Training Strategy:

- Sử dụng bộ ba dữ liệu $\langle q, d^+, d^- \rangle$.
- **Cross-entropy loss:** Tối ưu hóa các tham số để mô hình ưu tiên các tài liệu liên quan hơn các tài liệu không liên quan.

Nhu cầu về dữ liệu huấn luyện:

- Cần bộ ba: Query + Relevant (d^+) + Irrelevant (d^-).
- **Gold Positives**: Khai thác từ các tập dữ liệu chuẩn (ví dụ: MS MARCO).

Nhu cầu về dữ liệu huấn luyện:

- Cần bộ ba: Query + Relevant (d^+) + Irrelevant (d^-).
- **Gold Positives**: Khai thác từ các tập dữ liệu chuẩn (ví dụ: MS MARCO).

Chiến lược tạo Negative Examples:

- Thực hiện lấy mẫu ngẫu nhiên từ danh sách top-1000 của một hệ thống IR sẵn có.
- Mục tiêu: Tìm các tài liệu có độ tương đồng bề mặt cao nhưng không chứa thông tin trả lời đúng.

Giải pháp khi thiếu positive examples:

- Tận dụng các **short answer strings** để dán nhãn tạm thời.

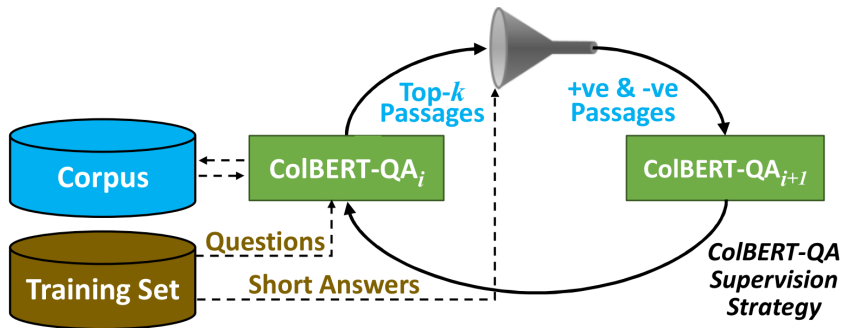
Giải pháp khi thiếu positive examples:

- Tận dụng các **short answer strings** để dán nhãn tạm thời.

Quy trình lặp:

- ➊ Dùng hệ thống IR hiện có để khai thác dữ liệu:
 - **Positives**: Top kết quả có chứa chuỗi đáp án.
 - **Negatives**: Top kết quả không chứa chuỗi đáp án.
- ➋ Huấn luyện một **retriever** mới từ dữ liệu này.
- ➌ Sử dụng bộ mới để lặp lại quy trình nhằm tăng độ chính xác.

Relevance-guided Supervision



Thách thức về khả năng scale:

- Hệ thống phải xếp hạng độ tương đồng cho mọi tài liệu khả thi so với truy vấn.
- **Sparse Vectors**: Sử dụng **Inverted Index** - cơ chế này đã được tối ưu hóa cực tốt trong nhiều thập kỷ.
- **Dense Vectors**: Đưa về bài toán **Nearest Neighbor Search**.

Giải pháp tìm kiếm:

- **Exact Search:** Việc tính tích vô hướng với từng vector trong hàng triệu tài liệu là quá chậm.
- **Approximate Nearest Neighbor:** Tìm kiếm láng giềng gần đúng giúp tăng tốc độ xử lý lên gấp nhiều lần.

Thư viện Faiss:

- Được phát triển bởi Facebook AI Research.
- Chứa các thuật toán tối ưu để tìm kiếm và phân cụm các vector dày đặc.
- Hỗ trợ chạy trên cả CPU và GPU để đạt hiệu năng tối đa.

- 1 Information Retrieval with Dense Vectors
- 2 Retrieval-Augmented Generation (RAG)**
- 3 Datasets
- 4 Evaluating Question Answering

Tại sao cần RAG?

LLM có thể gặp

- Hallucination
- Knowledge outdated
- Không truy cập dữ liệu riêng

Ví dụ

Hỏi chatbot nội bộ công ty:

“Chính sách nghỉ phép năm 2026 là gì?”

LLM thuần có thể:

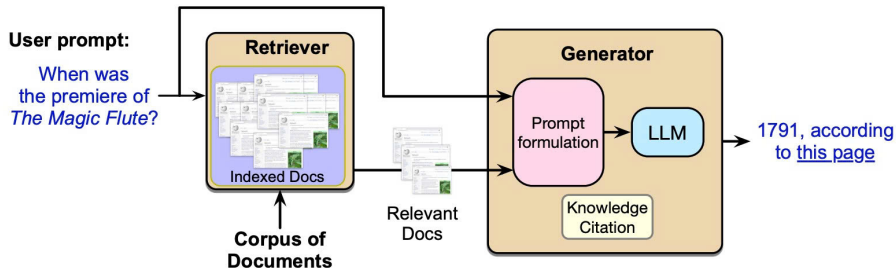
- Không biết
- Trả lời sai

Ý tưởng chính của RAG

$\text{RAG} = \text{Retrieval} + \text{Generation}$

Thay vì bắt model phải **nhớ tất cả**, ta cho model **truy xuất tài liệu liên quan** rồi mới sinh câu trả lời.

Pipeline RAG



- 1 Retrieve top- k docs từ tập tài liệu D
- 2 Ghép docs với câu hỏi thành prompt
- 3 Đưa vào LLM để generate answer

Công thức

$$R(q) = d_1, d_2, \dots, d_k$$

Công thức của RAG

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i \mid R(q), q, x_{<i})$$

Ký hiệu

- q : Câu hỏi của user
- $R(q)$: Documents retrieve được
- x_i : Token thứ i
- $x_{<i}$: Các token trước đó

Ý nghĩa

Khi sinh token tiếp theo, model dựa trên:

- Query
- Tài liệu retrieve
- Ngữ cảnh đã sinh

Nhược điểm của RAG

- Retrieved documents có thể nhiễu
- Một lần retrieve có thể chưa đủ
- Không phải lúc nào cũng cần retrieve

1. Agent-based RAG

Có thể chọn:

- Có retrieve hay không
- Retrieve từ nguồn nào

2. Reranker

Sắp xếp lại documents sau retrieval để:

- Giảm noise
- Chọn passage tốt hơn

3. Multi-hop Retrieval

Retrieve nhiều bước cho câu hỏi phức tạp.

- 1 Information Retrieval with Dense Vectors
- 2 Retrieval-Augmented Generation (RAG)
- 3 Datasets**
- 4 Evaluating Question Answering

Phân loại theo mục đích sử dụng ban đầu:

- **Natural Questions:**

- *Natural Questions (Google)*: Truy vấn ẩn danh từ người dùng thực tế trên Google Search.
- *MS MARCO (Bing)*: 1 triệu câu hỏi từ Bing logs kèm câu trả lời do con người tạo ra.
- Đặc điểm: Phản ánh cách đặt câu hỏi tự nhiên của con người.

QA Datasets Landscape: Natural vs. Probing

Phân loại theo mục đích sử dụng ban đầu:

• Natural Questions:

- *Natural Questions (Google)*: Truy vấn ẩn danh từ người dùng thực tế trên Google Search.
- *MS MARCO (Bing)*: 1 triệu câu hỏi từ Bing logs kèm câu trả lời do con người tạo ra.
- Đặc điểm: Phản ánh cách đặt câu hỏi tự nhiên của con người.

• Probing Questions:

- *MMLU (Massive Multitask Language Understanding)*: 15,908 câu hỏi thuộc 57 lĩnh vực.
- Nguồn: Lấy từ các kỳ thi.
- Mục tiêu: Kiểm tra trình độ chuyên gia và khả năng suy luận.

Mở rộng ra ngoài biên giới tiếng Anh:

- **DuReader**: Tài nguyên QA tiếng Trung khổng lồ dựa trên search engine và cộng đồng hỏi đáp.
- **TyDi QA**: Bộ dữ liệu bao gồm 204K cặp QA từ 11 ngôn ngữ đa dạng (Ả Rập, Bengal, Thái, Nga...).

Task Dimensions: Open-book vs. Closed-book

Nguồn gốc từ bài toán Đọc hiểu:

- AI nhận một câu hỏi và một văn bản, nhiệm vụ là trích xuất câu trả lời từ đó.

Task Dimensions: Open-book vs. Closed-book

Nguồn gốc từ bài toán Đọc hiểu:

- AI nhận một câu hỏi và một văn bản, nhiệm vụ là trích xuất câu trả lời từ đó.

Phân loại theo cơ chế truy xuất:

- **Open-book QA:** Hệ thống được cung cấp các tài liệu bổ trợ, ví dụ thông qua cơ chế **RAG**.
- **Closed-book QA:** AI trả lời trực tiếp từ kiến thức nội tại, không có thành phần truy xuất.

Task Dimensions: Open-book vs. Closed-book

Nguồn gốc từ bài toán Đọc hiểu:

- AI nhận một câu hỏi và một văn bản, nhiệm vụ là trích xuất câu trả lời từ đó.

Phân loại theo cơ chế truy xuất:

- **Open-book QA:** Hệ thống được cung cấp các tài liệu hỗ trợ, ví dụ thông qua cơ chế **RAG**.
- **Closed-book QA:** AI trả lời trực tiếp từ kiến thức nội tại, không có thành phần truy xuất.

Ví dụ điển hình:

- *Natural Questions*: Có thể là *Open-book* hoặc *Closed-book*.
- *MMLU*: Hoàn toàn là *Closed-book*.

Task Dimensions: Formats and Prompting

Answer Format:

- **Multiple-choice:** Trắc nghiệm.
- **Freeform:** Tự luận/Tự do.

Task Dimensions: Formats and Prompting

Answer Format:

- **Multiple-choice:** Trắc nghiệm.
- **Freeform:** Tự luận/Tự do.

Chiến lược Prompting:

- **Zero-shot:** Chỉ đưa ra câu hỏi và yêu cầu câu trả lời ngay lập tức.
- **Few-shot:** Đưa ra câu hỏi kèm theo một vài ví dụ minh họa trước khi đặt câu hỏi thật.

- 1 Information Retrieval with Dense Vectors
- 2 Retrieval-Augmented Generation (RAG)
- 3 Datasets
- 4 Evaluating Question Answering

Evaluating QA: Exact Match

Đánh giá dựa trên sự khớp nối tuyệt đối:

- Áp dụng cho **multiple-choice questions**.
- **Định nghĩa**: Tỷ lệ phần trăm câu trả lời dự đoán trùng khớp hoàn toàn với **gold answer**.
- Công thức đơn giản:

$$EM = \frac{\text{Số câu khớp hoàn toàn}}{\text{Tổng số câu hỏi}} \times 100\% \quad (1)$$

Evaluating QA: Exact Match

Đánh giá dựa trên sự khớp nối tuyệt đối:

- Áp dụng cho **multiple-choice questions**.
- **Định nghĩa**: Tỷ lệ phần trăm câu trả lời dự đoán trùng khớp hoàn toàn với **gold answer**.
- Công thức đơn giản:

$$EM = \frac{\text{Số câu khớp hoàn toàn}}{\text{Tổng số câu hỏi}} \times 100\% \quad (1)$$

Đánh giá cho câu trả lời tự do:

- Đo lường mức độ trùng lặp chuỗi giữa dự đoán và tham chiếu.
- **Cơ chế Bag of Tokens:** Coi văn bản là một túi chứa các từ, không quan tâm đến thứ tự.

Evaluating QA: Token F1 Score

Đánh giá cho câu trả lời tự do:

- Đo lường mức độ trùng lặp chuỗi giữa dự đoán và tham chiếu.
- **Cơ chế Bag of Tokens:** Coi văn bản là một túi chứa các từ, không quan tâm đến thứ tự.

Quy trình tính toán:

- 1 Tính Precision và Recall dựa trên số lượng tokens trùng lặp.
- 2 Tính F1 score cho từng câu hỏi cụ thể.
- 3 Trả về **Average F1** trên toàn bộ tập dữ liệu.